

MongoDB Indexing Assignment

Collection used: **restaurants** | Database: **restaurantDB** | Tool: mongosh

Name: _____ Roll No.: _____ Date: _____

Note: The outputs below are the expected results for the sample data in Question 1. Exact millisecond values depend on your machine, so run the commands in mongosh and replace the timings with your own if you want to submit real readings.

Question 1: Create the collection and find Italian restaurants

Create a MongoDB collection called 'restaurants', insert at least 10 documents with the fields name, cuisine, location and rating, and run a find query to retrieve all restaurants with cuisine 'Italian'.

Commands

```
use restaurantDB
db.createCollection("restaurants")

db.restaurants.insertMany([
  { name: "Bella Napoli",      cuisine: "Italian", location: "Ahmedabad", rating: 4.5 },
  { name: "La Piazza",        cuisine: "Italian", location: "Surat",      rating: 4.2 },
  { name: "Olive Corner",     cuisine: "Italian", location: "Vadodara",  rating: 4.0 },
  { name: "Dragon Wok",       cuisine: "Chinese", location: "Ahmedabad", rating: 4.3 },
  { name: "Golden Chopsticks", cuisine: "Chinese", location: "Ahmedabad", rating: 4.1 },
  { name: "Mandarin House",   cuisine: "Chinese", location: "Surat",     rating: 3.9 },
  { name: "Pizza Palace",     cuisine: "Pizza",   location: "Surat",     rating: 4.6 },
  { name: "Wood Fire Hub",    cuisine: "Pizza",   location: "Surat",     rating: 4.5 },
  { name: "Slice Junction",   cuisine: "Pizza",   location: "Surat",     rating: 4.4 },
  { name: "Oven Story",       cuisine: "Pizza",   location: "Surat",     rating: 4.3 },
  { name: "Cheesy Bites",     cuisine: "Pizza",   location: "Surat",     rating: 4.0 },
  { name: "Dough Brothers",   cuisine: "Pizza",   location: "Surat",     rating: 3.7 },
  { name: "Crust & Co",        cuisine: "Pizza",   location: "Ahmedabad", rating: 4.2 },
  { name: "Spice Route",      cuisine: "Indian",  location: "Rajkot",    rating: 4.7 },
  { name: "Thali House",      cuisine: "Gujarati", location: "Ahmedabad", rating: 4.8 },
  { name: "Taco Fiesta",      cuisine: "Mexican", location: "Surat",     rating: 3.8 }
])

db.restaurants.find({ cuisine: "Italian" })
```

Output (16 documents inserted, 3 Italian restaurants returned)

name	cuisine	location	rating
Bella Napoli	Italian	Ahmedabad	4.5
La Piazza	Italian	Surat	4.2
Olive Corner	Italian	Vadodara	4.0

Question 2: Single field index on 'cuisine' and performance comparison

Add a single field index on 'cuisine' using `createIndex()`. Measure and compare the query execution time for finding all 'Italian' restaurants before and after adding the index.

Commands

```
// BEFORE the index
db.restaurants.find({ cuisine: "Italian" }).explain("executionStats")

// Create the index
db.restaurants.createIndex({ cuisine: 1 })
```

```
// AFTER the index
db.restaurants.find({ cuisine: "Italian" }).explain("executionStats")
```

Comparison of the two explain() results

Metric	Before index	After index
Winning plan stage	COLLSCAN (full collection scan)	FETCH -> IXSCAN (index cuisine_1)
totalKeysExamined	0	3
totalDocsExamined	16	3
nReturned	3	3
executionTimeMillis	about 0-1 ms	about 0-1 ms

Explanation: Before the index, MongoDB performs a collection scan and reads all 16 documents to return 3 matches. After the index, it looks up 'Italian' in the index and fetches only the 3 matching documents. With only 16 documents the time difference is too small to measure (0-1 ms), so the real improvement is seen in the number of documents examined. On a large collection (for example, 1 lakh or more documents) this reduction translates directly into much faster queries.

Question 3: Compound index on 'cuisine' and 'location'

Create a compound index on 'cuisine' and 'location'. Write a query to find all 'Chinese' restaurants in 'Ahmedabad' and check whether the compound index is used by the query planner.

Commands

```
db.restaurants.createIndex({ cuisine: 1, location: 1 })

db.restaurants.find({ cuisine: "Chinese", location: "Ahmedabad" })
    .explain("executionStats")
```

Result: Dragon Wok (4.3) and Golden Chopsticks (4.1). Key part of the explain() output:

```
winningPlan: {
  stage: "FETCH",
  inputStage: {
    stage: "IXSCAN",
    keyPattern: { cuisine: 1, location: 1 },
    indexName: "cuisine_1_location_1",
    indexBounds: {
      cuisine: [ '['Chinese', 'Chinese']' ],
      location: [ '['Ahmedabad', 'Ahmedabad']' ]
    }
  }
}
executionStats: { nReturned: 2, totalKeysExamined: 2, totalDocsExamined: 2 }
```

Conclusion: Yes, the query planner uses the compound index. The winning plan shows an IXSCAN on cuisine_1_location_1, and only 2 keys and 2 documents were examined, exactly the number returned. The planner prefers it over the single 'cuisine' index because it narrows the search using both fields.

Question 4: Drop the 'cuisine' index and observe performance

Drop the index created on 'cuisine' and observe how the query performance changes when searching for a specific cuisine. Record observations in 2-3 lines.

Commands

```
db.restaurants.dropIndex("cuisine_1")

db.restaurants.find({ cuisine: "Italian" }).explain("executionStats")
```

Observations

1. After dropping `cuisine_1`, the query on cuisine 'Italian' still uses an IXSCAN, because 'cuisine' is the first (leftmost) field of the compound index `cuisine_1_location_1`. It still examines only 3 documents, so performance stays almost the same.
2. This shows the single-field index was redundant once the compound index existed. Only if the compound index is dropped as well would the query fall back to a COLLSCAN and examine all 16 documents.

Question 5: Top 5 highest rated 'Pizza' restaurants in 'Surat'

Generate a MongoDB query that efficiently fetches the top 5 highest rated 'Pizza' restaurants in 'Surat', using relevant indexes. Paste the query and explain in one line how indexing helps.

Generated query

```
// Index: equality fields first (cuisine, location), sort field last (rating)
db.restaurants.createIndex({ cuisine: 1, location: 1, rating: -1 })

// Query
db.restaurants.find({ cuisine: "Pizza", location: "Surat" })
    .sort({ rating: -1 })
    .limit(5)
```

Expected result

name	rating
Pizza Palace	4.6
Wood Fire Hub	4.5
Slice Junction	4.4
Oven Story	4.3
Cheesy Bites	4.0

How indexing helps (one line): The compound index already stores the Pizza + Surat entries sorted by rating in descending order, so MongoDB reads just the first 5 entries from the index without scanning the whole collection or doing an in-memory sort.